Local Accuracy Is Not a Contract for Abstract Transformers

Wenrui Liang, Huijie Xu, Zhibo Jin, Zhiyu Zhu, Hongyang Chen, Biaoliang Chen, and Jianlong Zhou

Abstract. Static analyzers prove safety properties by repeatedly applying abstract transformers until they reach an invariant covering every reachable state. Precise transformers are hard to write by hand, so learned substitutes are attractive, though usually judged by accuracy on sampled inputs. But an analysis feeds each output back as its next input, so its result depends only on the few inputs its iteration visits, while accuracy weighs all inputs equally. Classical soundness conditions should be checked instead, but it is unclear which a learned model must meet, which only its final result must meet, and with what evidence. We propose role-specific admission contracts, which assign each condition to its responsible component and state which object a passed check admits. Exhaustive checks in a finite interval domain show that highly accurate models can yield unsound invariants, while models violating laws required on every input can yield sound ones. Even an error on the iteration path matters only if it stops iteration early. Trust must therefore come from the laws or the result. Combining the model with the exact operator enforces the laws but calls it at every step. Retraining on counterexamples avoids those calls and yields more sound invariants, but most retrained models fail an offline check of every input. A passed post-fixpoint check certifies the final invariant with one call to the exact operator, even if the model violates the laws. Evaluations should say whether a model or a result is trusted and report accuracy, these laws, and soundness separately.

Keywords: Abstract interpretation · Learned abstract transformers · Admission contracts · Post-fixpoint certification

1 Introduction

Abstract transformers carry out each semantic step of a static analysis, and transformers that are both sound and precise are hard to write by hand. Comparison with synthesized transformers [12] and differential testing [13] have found unsound transformers in existing analyzers, and testing has found imprecise ones in a production compiler [18]. Rather than writing transformers by hand, one can synthesize them from the concrete semantics [12] or learn analysis rules from data [1], and a learned model can likewise serve as a transformer. Learned components are usually evaluated on sampled inputs: neural interval and octagon transformers by the fraction of sound outputs on a random test set [9], and models that replicate data-flow analyses by precision and recall [5]. A transformer,

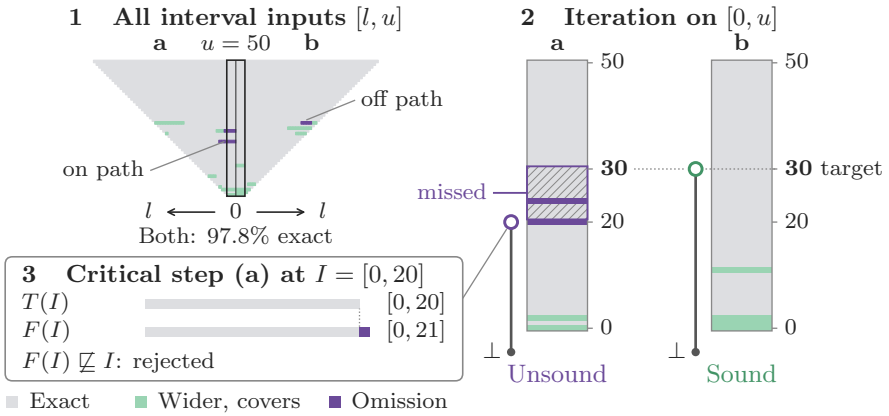


Fig. 1. Two models with equal full-domain accuracy reach opposite conclusions ($N = 50$, $K = 30$). (1) Both are exact on 97.8% of all inputs. Framed columns are the inputs $[0, u]$ visited by iteration. (2) Model a hits an omission on this path and stops at $[0, 20]$, while model b's omissions lie off the path and it reaches $[0, 30]$. (3) At $I = [0, 20]$, model a returns $T(I) = I$, but the exact operator gives $F(I) = [0, 21] \not\sqsubseteq I$, so I is rejected.

however, is not applied just once: the analysis keeps applying it to the states accumulated so far until they stabilize.

A prediction error and an analysis failure are therefore different things. Figure 1 shows this for the loop $x := 0$; **while** $x < 30$ **do** $x := x + 1$. Two trained models are almost equally accurate, each matching the exact operator on nearly all held-out inputs. One of them stops at $[0, 20]$, so an analyzer that trusted it would report $x \leq 20$ at the loop head, although values up to 30 are reachable. The other omits states only on inputs that its iteration never visits, and reaches the sound invariant $[0, 30]$. Accuracy thus settles neither whether a model can be reused as a component nor whether one result can be trusted.

The soundness conditions themselves are classical: abstract interpretation already shows that an operator covering the exact one yields a sound invariant once iteration stabilizes, and that an invariant closed under the exact operator is sound [3, 4]. Existing work enforces these conditions one object at a time: it verifies proposed invariants [2, 7, 19] or imposes semantic constraints when synthesizing transformers [12]. For a learned component, what remains open is which of these conditions it must meet, and with what evidence. We answer this with *role-specific admission contracts*, which turn each condition into an obligation of the component responsible for it and name the object that a passed check admits.

The contracts separate three things that a single accuracy score conflates, and these come apart because each looks at different inputs. Local accuracy averages over sampled inputs. Two *global laws*, coverage and monotonicity, must hold on every input, although iteration never visits most of them. Whether one result is sound depends only on the inputs that iteration visits: it is unsound only

if iteration stops at an input on which the model omits states. Trust can thus be established at two points (Figure 2), by three routes that differ in how they use the exact operator, which we call the *oracle*. Before iteration, a candidate operator is admitted for every input. An envelope combines the model with the oracle and satisfies the global laws by construction, but it needs the oracle on every input and so serves as a baseline. Counterexample-guided retraining instead tries to make the model satisfy the laws itself. After iteration, a post-fixpoint check certifies the one invariant produced. To check every input, we work in a finite interval domain and study two kinds of candidates: *planted patterns* with known defects, and trained models.

Each finding is therefore an exact witness rather than a sampled estimate. The two most accurate models, those of Figure 1, violate both global laws. Whether an omission on the iteration path stops iteration depends on the loop: an upper bound one short stops iteration on a loop that steps by one, but not on one that steps by two. Exact-match accuracy penalizes a safe error, an output wider than the oracle's, as heavily as an omission, and closer fits can turn safe errors into omissions. Retraining on counterexamples lets many more results pass the post-fixpoint check, because acceptance depends on only a few inputs, but most retrained models still violate a global law. Across the routes, a guarantee for every input needs the oracle on every input, whereas certifying one result needs it only once. Evaluations should therefore state whether they trust a model or a result, and report accuracy, the global laws, and the soundness of each result separately. Our contributions are (i) role-specific admission contracts and the evidence for each obligation (Section 2, Table 1); (ii) exact witnesses that accuracy predicts neither the global laws nor the soundness of a result, and the mechanisms behind them (Section 3); (iii) a comparison of three routes to trust by what each certifies and the oracle access it needs (Figure 2, Appendix C); and (iv) this evaluation protocol and a research agenda (Section 5).

2 Contracts and Admission Scope

Setting. Let $\mathcal{A}_N = \{\bot\} \cup \{[l,u] \mid 0 \le l \le u \le N\}$, ordered by containment. Concretization γ gives the interval's integer members, and $\gamma(\bot) = \varnothing$. Join is the interval hull, $\bot \sqcup a = a$, and $\top = [0,N]$. The carrier $\mathcal{A}_N$, which we call the full domain, has $m = 1 + (N+1)(N+2)/2$ elements. For $1 \le K \le N$, the concrete loop functional $f(S) = \{0\} \cup \{x+1 \mid x \in S, x < K\}$ adds the entry state and the guarded successors. Its exact interval abstraction, called the *oracle*, is

$$F(a) = [0,0] \sqcup \big((a \sqcap [0,K-1]) + 1\big). \tag{1}$$

The oracle is the prediction target. Thus $F(\bot) = [0,0]$, F never returns bottom, and the values reachable at the loop head form $[0,K]$. Here "exact" means the best interval image of one semantic step [4], which need not equal the reachable set. An analysis iterates a candidate $X : \mathcal{A}_N \to \mathcal{A}_N$ from bottom with the *joined update* $U_X(a) = a \sqcup X(a)$, which retains the accumulated states and terminates because the domain has finite height.

Table 1. Role-specific admission contracts and the evidence for each in this paper.

Role	Obligation	Evidence
Loop functional X	coverage, $F(a) \sqsubseteq X(a)$ for all a	envelope or full-domain check
Joined update U_X	retention, $a \sqsubseteq U_X(a)$	holds by definition
Final invariant I	closure, $F(I) \sqsubseteq I$	post-fixpoint check
Least solution	monotonicity of X	envelope or full-domain check

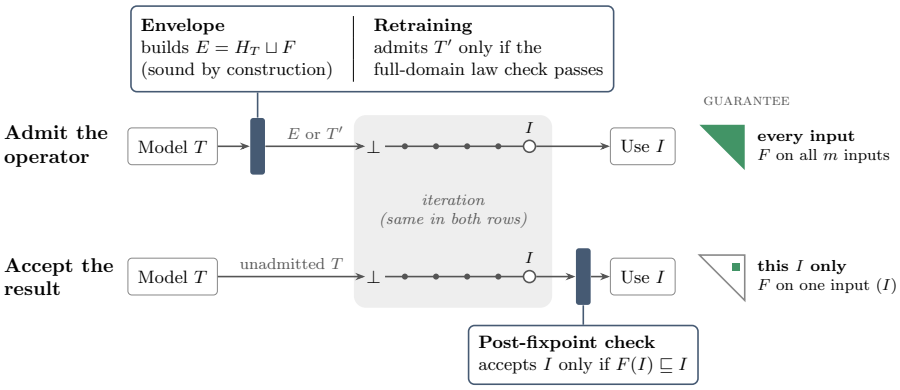


Fig. 2. Two checkpoints at which the three routes establish trust. Top: before iteration, the envelope admits E by construction, querying F on every input at inference, and retraining admits T' only after an offline check of every input. Bottom: the post-fixpoint check certifies only the final I of an unadmitted T, including a retrained T' that fails its check. Green marks what each guarantee covers, and m is the number of inputs.

Contracts. In Table 1, the learned model plays the role of the loop functional X. Two properties of X are checked on every input and called *global laws*: *coverage*, $F(a) \sqsubseteq X(a)$, and *monotonicity*, $X(a) \sqsubseteq X(b)$ whenever $a \sqsubseteq b$. An input on which coverage fails is an *omission*. A candidate has a *coverage pass* when it has no omission and a *global pass* when both laws hold on every input. Coverage alone makes joined iteration sound: a stable joined result I satisfies $F(I) \sqsubseteq X(I) \sqsubseteq I$, so $\gamma(I)$ contains the entry state and is closed under concrete transitions, and induction gives *Reach* $\subseteq \gamma(I)$. The same argument shows that any I with $F(I) \sqsubseteq I$ is sound. Hence if a stable joined result I is unsound, then $F(I) \not\sqsubseteq I$ although $X(I) \sqsubseteq I$, so I is an omission at which iteration stopped. Monotonicity instead governs the precision of least fixpoints and some iteration algorithms.

Admitting the operator by construction: the envelope. The envelope (Figure 2, top) turns any candidate into an operator that satisfies both global laws. For a trained model or planted pattern T, define

$$H_T(a) = \bigsqcup_{b \sqsubseteq a} T(b), \qquad E(a) = H_T(a) \sqcup F(a). \tag{2}$$

The downward join supplies monotonicity and the oracle term supplies coverage, so the result cannot be more precise than the oracle:

Proposition 1. *E is monotone and $F \sqsubseteq E$. If $T(b) \sqsubseteq F(b)$ for every b, then $E = F$.*

Proof. Increasing a enlarges the downward set, and F is monotone. The explicit oracle term gives coverage. Under the hypothesis, $b \sqsubseteq a$ implies $T(b) \sqsubseteq F(b) \sqsubseteq F(a)$, so $H_T(a) \sqsubseteq F(a)$. □

The guarantee is a property of E, and T itself may still violate both laws.

Admitting the operator by checking: retraining. Counterexample-guided retraining, in the spirit of counterexample-guided data augmentation [6], aims at a model that satisfies the global laws itself, so that inference needs no oracle (Figure 2, top). Each round adds oracle labels for omission inputs and violating pairs, retrains, and checks the full domain again, stopping on a global pass, on its round budget, or on stagnation (Appendix A), and any guarantee comes only from this check.

Accepting one result: the post-fixpoint check. If only the final invariant needs a guarantee, closure can be checked directly (Figure 2, bottom):

$$\mathsf{Cert}(I) \text{ accepts} \iff F(I) \sqsubseteq I. \tag{3}$$

Acceptance certifies I, not X, needs one oracle call after iteration, and implies soundness; rejection need not imply unsoundness (Section 5).

3 Evidence from Trained Models

Design. Unless stated otherwise, accuracy means held-out exact match. The primary grid trains 45 models at $N = 30$, $K = 18$. A second grid of 20 models at $N = 50$, $K = 30$, specified after the primary grid's results and before its own runs, supplies further witnesses rather than a confirmatory test. A second loop, which steps by two (Section 5), repeats the primary grid; it was run after the main results with its analysis plan fixed before training (Appendix D). All models remain in the analysis. Retraining is applied to a fixed cohort of 20 models (Appendix A). No model predicts an upper bound above K, the largest training label, so on the main loop every envelope and every accepted result is the exact invariant $[0, K]$ (Appendix B). The routes therefore differ in precision only on single inputs (Table 4).

Accuracy does not identify a sound result. The two most accurate models give opposite answers at the program level (Figures 1 and 3). At 97.0% held-out exact match, model a stabilizes at $[0, 20]$, and the post-fixpoint check rejects its result. At 96.5%, model b reaches $[0, 30]$ and is accepted. Neither satisfies either global law. By the fraction of sound outputs, the measure of Gomber and Singh [9],

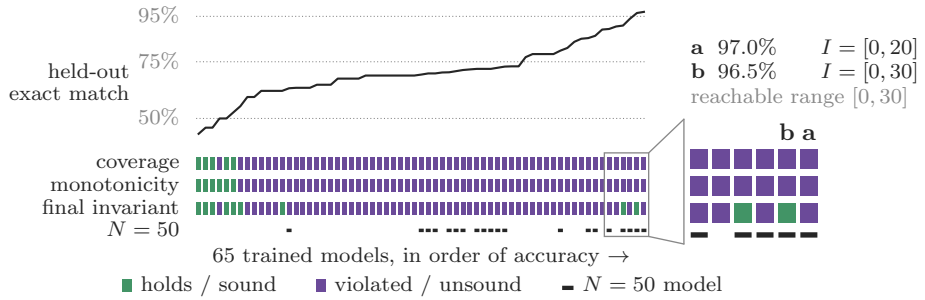


Fig. 3. All 65 trained models, sorted by held-out exact match (curve). Each column shows whether a model passes coverage and monotonicity and whether its final invariant is sound. The bottom row marks the $N = 50$ grid.

Table 2. Results under joined updates. Arrows show the same 10 models before and after retraining. The last row is the second loop (Appendix D).

N	Cohort	Count	Coverage pass	Global pass	Accept	Accepted with violation
30	Trained	45	5/45	5/45	7/45	2
50	Trained	20	0/20	0/20	2/20	2
30	Retrained (before → after)	10	2 → 5	2 → 3	2 → 7	0 → 4
50	Retrained (before → after)	10	0 → 0	0 → 0	0 → 3	0 → 3
30	Trained, second loop	45	4/45	2/45	24/45	22

model a scores 99.5%. Its verdict turns on a single input: its path from $\perp$ to $[0, 20]$ visits 20 of the 1327 inputs (Appendix B) and ends at one of its six omissions. On the second loop, sound and unsound results again overlap in accuracy, and the three most accurate models violate both laws yet are sound. The planted patterns show the same separation with a known cause (Appendix B).

An omission matters only if it stops iteration. Like F, every trained model outputs the lower bound 0 on nonempty inputs, so every omission is an upper bound that falls short. An omission off the iteration path is harmless, and paths are short (Appendix B). Of the 11 models accepted despite a law violation (Table 2), nine have omissions only off their paths, and the other two violate only monotonicity. On the path, an omission stops iteration only if the output stays inside the current interval. In the main loop F raises the upper bound by one, so a bound one short already stops iteration, and every model with a path omission is unsound. In the second loop F raises it by two, so a bound one short does not stop iteration: 30 of the 51 path omissions are passed over, and 12 models are sound despite them.

Closer fits turn safe errors into omissions. Exact match penalizes a wider output as heavily as an omission. In the primary grid, the median share of inexact outputs that are omissions rises from 0% at width 16 and 20% coverage to 30% at width 64 and 80% (Table 3). The $N = 50$ grid does not follow this pattern: its most accurate configuration, which produced models a and b, has the lowest share, 9%. Each of the 30 primary-grid models at 50% or 80% coverage falls one short on some input $[0, u]$ that exact iteration visits, and none is accepted. On the second loop, width 64 at 80% coverage, the most accurate configuration there, gives five sound models out of five (Table 6). Five of the seven accepted models output $[0, K]$ on every input. Before retraining, these constant models are the only main-loop models that satisfy both global laws, and the second loop's two are nearly constant. Of the 38 unsound primary-grid models, 21 stop at a training input; for these, the fatal omission is a failure to fit.

Retraining improves acceptance more than global pass rates. Retraining raises acceptance from 2/20 to 10/20 but global passes only from 2/20 to 3/20, because the two depend on different inputs. Each newly accepted model repairs the input at which it stopped, but at most 7 of the 14 to 56 omissions it repairs lie on its new path. The global laws depend on every input, and retraining makes the same trade as above: most new omissions lie on or next to an added input and were wider before. Most models that still violate a law end with nearly all violating inputs already in their training data. These results suggest that fitting the available examples remains a bottleneck. The counterexample order barely matters (Appendix A).

4 Related Work

Evidence on sampled inputs. Gomber and Singh [9] train neural interval and octagon transformers, report the fraction of sound outputs, and check candidate invariants with an SMT solver; their suggested fallback to hand-crafted transformers on unsound outputs, like our envelope, needs a trusted operator on every input it guards. We do not reproduce their models or training; we add role-specific guarantee scopes and exact witnesses of their separation.

Admitting the operator by construction. Kalita et al. [12] synthesize sound transformers from the concrete semantics, optimizing precision within a specified language, so their semantic constraints discharge coverage. Gomber et al. [8] search a parametric space of sound outputs with gradients. Both build the guarantee into the producer, whereas our envelope wraps an ordinarily trained model.

Admitting the operator by checking. Gu et al. [10] synthesize transformers with language models and accept only candidates whose soundness a solver verifies for all inputs, a symbolic counterpart of our full-domain check. Bielik et al. [1] refine learned analysis rules with counterexamples, and Sivaraman et al. [17] learn monotonic networks from them. Our retraining applies the same loop to both global laws (Section 3).

Accepting one result. ICE [7] learns inductive invariants from positive, negative, and implication examples, Lemur [19] combines language models with automated reasoners in a sound calculus, and Chakraborty et al. [2] reduce verifier queries by ranking generated invariants. Each proposes an invariant and then checks it, as our post-fixpoint route does, whereas we also ask when its producer can be admitted as a reusable operator.

Other roles. Oh et al. [15] learn how to adapt a program analysis, and He et al. [11] how to simplify numerical abstract states soundly; both choose among sound operations, so no coverage obligation arises (Table 1).

5 Implications, Scope, and Conclusion

An evaluation protocol. A global failure need not prevent using a model to propose an invariant, but an accepted invariant does not license reusing the model unchecked. Evaluations should therefore name the role and the object a guarantee covers, report each global law and each post-fixpoint decision separately, credit precision only for sound results, and separate the oracle costs of training, full-domain checks, and post-fixpoint queries.

Certification is sound but incomplete. Rejection need not imply unsoundness. Every sound interval passes the check in our main loop, but for $x := 0$; **while** $x < 18$ **do** $x := x + 2$ the sound $I = [0, 18]$ is rejected: it contains the unreachable value 17, whose successor 19 lies outside I. No model trained on this loop stops there, since every one maps $[0, 18]$ to $[0, 19]$, so this case is shown by construction (Appendix D).

Scope. Our evidence consists of exact witnesses in two exhaustively checked loop families, obtained with few seeds and shared data partitions, so it shows that the separations occur but not how often.

A research agenda. First, where domains cannot be enumerated, which evidence can establish global laws for a learned operator, symbolically or by construction [8, 10, 12]? Second, where exact operators such as relational joins are expensive [9, 14, 16], when does admission repay its cost, and how often are sound results rejected? Finally, which obligations do widening and composed learned operators add, and could learned choices among sound operators [11, 15] reach the same goals without learning semantics?

Conclusion. Admission should state which object a guarantee covers. A learned component can be trusted as a reusable operator, through the evidence its contract requires, or for one result, through a passed post-fixpoint check, but not through accuracy alone.

Statement on the Use of Large Language Models. Generative AI tools assisted with manuscript refinement, language clarity and readability, and LaTeX formatting. The authors are responsible for the final manuscript.

Data Availability Statement. The code, trained models, experimental results, and reproduction scripts are publicly available at https://github.com/qichiovo/ifs2027-nier-artifact/releases/tag/v1.0.

References

1. Bielik, P., Raychev, V., Vechev, M.: Learning a static analyzer from data. In: CAV, Part I. LNCS, vol. 10426, pp. 233–253. Springer (2017). https://doi.org/10.1007/978-3-319-63387-9_12
2. Chakraborty, S., Lahiri, S., Fakhoury, S., Lal, A., Musuvathi, M., Rastogi, A., Senthilnathan, A., Sharma, R., Swamy, N.: Ranking LLM-generated loop invariants for program verification. In: Findings of the Association for Computational Linguistics: EMNLP 2023. pp. 9164–9175. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.614
3. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: POPL. pp. 238–252. ACM (1977). https://doi.org/10.1145/512950.512973
4. Cousot, P., Cousot, R.: Systematic design of program analysis frameworks. In: POPL. pp. 269–282. ACM (1979). https://doi.org/10.1145/567752.567778
5. Cummins, C., Fisches, Z.V., Ben-Nun, T., Hoefler, T., O'Boyle, M.F.P., Leather, H.: ProGraML: A graph-based program representation for data flow analysis and compiler optimizations. In: ICML. Proceedings of Machine Learning Research, vol. 139, pp. 2244–2253. PMLR (2021), https://proceedings.mlr.press/v139/cummins21a.html
6. Dreossi, T., Ghosh, S., Yue, X., Keutzer, K., Sangiovanni-Vincentelli, A., Seshia, S.A.: Counterexample-guided data augmentation. In: IJCAI. pp. 2071–2078 (2018). https://doi.org/10.24963/ijcai.2018/286
7. Garg, P., Löding, C., Madhusudan, P., Neider, D.: ICE: A robust framework for learning invariants. In: CAV. LNCS, vol. 8559, pp. 69–87. Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_5
8. Gomber, S., Banerjee, D., Singh, G.: Evolving abstract transformers for gradient-guided, adaptable abstract interpretation. Proceedings of the ACM on Programming Languages **10**(PLDI), 2474–2498 (2026). https://doi.org/10.1145/3808346
9. Gomber, S., Singh, G.: Neural abstract interpretation. In: VerifAI: AI Verification in the Wild, ICLR 2025 Workshop (2025), https://openreview.net/forum?id=WTyyhWhp4m
10. Gu, Q., Singh, A., Singh, G.: SAIL: Sound abstract interpreters with LLMs. Proceedings of the ACM on Programming Languages **10**(PLDI), 1534–1559 (2026). https://doi.org/10.1145/3808308
11. He, J., Singh, G., Püschel, M., Vechev, M.: Learning fast and precise numerical analysis. In: PLDI. pp. 1112–1127. ACM (2020). https://doi.org/10.1145/3385412.3386016
12. Kalita, P.K., Muduli, S.K., D'Antoni, L., Reps, T., Roy, S.: Synthesizing abstract transformers. Proceedings of the ACM on Programming Languages **6**(OOPSLA2), 1291–1319 (2022). https://doi.org/10.1145/3563334
13. Klinger, C., Christakis, M., Wüstholz, V.: Differentially testing soundness and precision of program analyzers. In: ISSTA. pp. 239–250. ACM (2019). https://doi.org/10.1145/3293882.3330553
14. Miné, A.: The octagon abstract domain. Higher-Order and Symbolic Computation **19**(1), 31–100 (2006). https://doi.org/10.1007/s10990-006-8609-1

15. Oh, H., Yang, H., Yi, K.: Learning a strategy for adapting a program analysis via Bayesian optimisation. In: OOPSLA. pp. 572–588. ACM (2015). https://doi.org/10.1145/2814270.2814309
16. Singh, G., Püschel, M., Vechev, M.: Fast polyhedra abstract domain. In: POPL. pp. 46–59. ACM (2017). https://doi.org/10.1145/3009837.3009885
17. Sivaraman, A., Farnadi, G., Millstein, T., Van den Broeck, G.: Counterexample-guided learning of monotonic neural networks. In: Advances in Neural Information Processing Systems. vol. 33 (2020), https://proceedings.neurips.cc/paper/2020/hash/8ab70731b1553f17c11a3bbc87e0b605-Abstract.html
18. Taneja, J., Liu, Z., Regehr, J.: Testing static analyses for precision and soundness. In: CGO. pp. 81–93. ACM (2020). https://doi.org/10.1145/3368826.3377927
19. Wu, H., Barrett, C., Narodytska, N.: Lemur: Integrating large language models in automated program verification. In: ICLR (2024), https://arxiv.org/abs/2310.04870

A Reproduction Details and Complete Configurations

The primary grid uses hidden widths 16, 32, 64, training coverage targets 20%, 50%, 80%, and seeds 0, 1, 2, 3, 4. The $N = 50$ grid uses the larger two widths, extreme coverages, and the same seeds. Both grids set $K = 0.6N$, which places the loop bound inside the domain and leaves values above K for over-approximation, and the coverage targets vary how much of the full domain a model sees during training. Each multilayer perceptron has two rectified-linear hidden layers, normalized interval endpoints and a bottom flag as input, and two endpoint classifiers. Legal decoding maximizes the sum of endpoint scores over $0 \leq l \leq u \leq N$. It uses neither F nor K. Adam uses learning rate 0.001, batch size 64, at most 300 epochs, and validation patience 30. Minimum validation loss selects checkpoints. The split for each seed reserves 15% for testing and 5% for validation, using floor counts, and nested training sets cover the stated fractions of the full domain.

Per seed, test and validation sets have 74 and 24 inputs at $N = 30$ and 199 and 66 at $N = 50$. Training sets for the 20%, 50%, and 80% targets have 99, 248, and 397 inputs at $N = 30$, and those for the 20% and 80% targets have 265 and 1061 inputs at $N = 50$, slightly below the targets because counts are floored. For each seed, training sets are nested prefixes of the pool left after selecting test and validation sets. Coverages use the full domain as denominator. The legal decoder selects the endpoint pair maximizing $s_l(i) + s_u(j)$ over $0 \leq i \leq j \leq N$, breaking ties in a fixed pair order. Bottom is an input but never an oracle output. Exact-match rates are stored separately for training, validation, test, and the full domain. Within each domain, all models share the loop semantics and evaluation procedure, and the hidden widths share the partitions of each seed. Monotonicity violations count comparable ordered pairs, including bottom and reflexive pairs.

Table 3 retains all seeds for every configuration. It reports descriptive minimum, median, and maximum values. Only models a and b exceed the prespecified 95% threshold, and no model reaches the higher thresholds of 98% and 99%. The primary grid's maximum is 90.5%.

The domain, oracle, and architecture stay fixed during retraining, and only data and parameters change. Each round retains earlier training data, and the reported model is the last completed round. The envelope uses m distinct oracle values, and the retraining checks reuse a cache of them. Inference with a retrained model uses only input encoding, saved neural parameters, and legal decoding. Each retraining run permits at most 5 rounds, each adding at most 64 unique inputs using deterministic counterexample order. Each round permits 100 epochs with validation patience 15. Retraining selects omission inputs in ascending order of state identifiers, then violating pairs lexicographically. Both previously unseen members count towards the input cap of the round, and a pair is selected only if they fit. Previously added inputs do not consume the allowance again. Training retains all original inputs. Because retraining may add original test inputs to the training data, we report no held-out accuracy for retrained models. A validation input added by retraining leaves the validation partition, and an empty remaining partition would stop the run. Early stopping uses its

Table 3. Configurations of all trained models. Accuracy is held-out exact match in percent, and counts use every seed of the stated configuration. Omission share is the median percentage of inexact inputs that are omissions.

N	Width	Cov. (%)	Median	Range	Omission share (%)	Global pass	Accept
30	16	20	50.0	43.2–55.4	0	3/5	4/5
30	16	50	64.9	62.2–78.4	13	0/5	0/5
30	16	80	67.6	59.5–77.0	10	0/5	0/5
30	32	20	59.5	45.9–68.9	3	2/5	2/5
30	32	50	68.9	62.2–78.4	17	0/5	0/5
30	32	80	68.9	63.5–81.1	17	0/5	0/5
30	64	20	64.9	62.2–68.9	17	0/5	1/5
30	64	50	73.0	67.6–85.1	26	0/5	0/5
30	64	80	83.8	68.9–90.5	30	0/5	0/5
50	32	20	69.8	63.3–71.9	17	0/5	0/5
50	32	80	85.4	71.9–89.4	22	0/5	0/5
50	64	20	71.9	69.8–72.9	23	0/5	0/5
50	64	80	94.0	70.9–97.0	9	0/5	2/5

loss. Runs stop after all checks pass, the round limit, or two rounds with no new counterexample input and no decrease in the combined violation count. Counts for each law remain separate. Figure 4 shows every recorded run, and no earlier round replaces a worse final round. The 20 retraining runs, which include models that already pass, complete 87 rounds in total. Each of the eight models that retraining brings to acceptance repairs the input at which it stopped. For three of them this input was already a training input and was never added as a counterexample. Three new paths extend the old ones, and five depart from them. Of the 526 omissions that retraining creates, 241 are added inputs, 184 lie next to one, with endpoints differing by at most one, and 426 were wider than F before. Of the 7791 monotonicity violations it creates, 66% involve an added input. Of the 17 runs that end with a law violation, 11 end with at most six violating inputs outside their training data. Shuffling the counterexample order, within omissions and within violating pairs or across both, with three seeds each, yields 9 to 11 accepted models and 3 or 4 global passes, against 10 and 3 under the fixed order. Only 3 of the 20 runs change their outcome with the order.

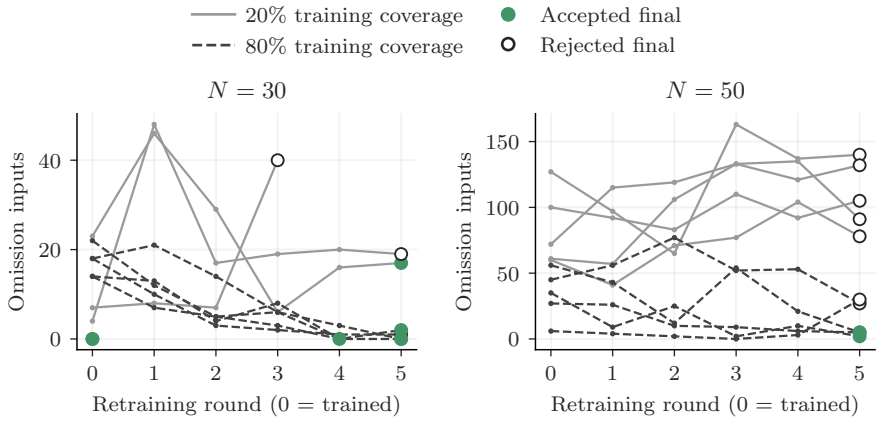


Fig. 4. Omission inputs through all retraining runs, with round zero as the trained model. Light-gray solid and dark-gray dashed lines indicate 20% and 80% training coverage, respectively. Filled green final markers indicate acceptance by the post-fixpoint check, and hollow markers indicate rejection (10/20 accepted). Already admitted models appear as isolated points. Lines end with the recorded run, and coincident final markers overlap.

B Patterns, Traces, and Envelope Precision

The width of $[l, u]$ is $u - l + 1$, and the width of bottom is zero. Recorded model upper endpoints never exceed K, because no training label does, so envelope iteration stays within $[0, K]$, and coverage forces equality at termination. The invariant $[0, K]$ preserves all true assertions $x \leq c$ for $K \leq c \leq N$. On the planted patterns, Proposition 1 gives $E = F$. The planted patterns use $N = 14$ and $K = 9$. The even pattern changes $F([0, K-1])$ to $[0, K-1]$ and $F([0, K])$ to $[0, K-2]$. The odd pattern changes only $F([0, K])$ to $[0, K-1]$. Both equal F elsewhere. On the full domain, the even pattern is exact on 119 of the 121 inputs, with 2 coverage omissions and 33 of 3181 comparable pairs violating monotonicity. The odd pattern is exact on 120, with 1 omission and 17 violating pairs. Under joined updates the even pattern stops at the unsound $[0, 8]$ and the odd pattern reaches the sound $[0, 9]$. A *direct update* $R_X(a) = X(a)$ instead replaces the accumulated states and can cycle. Under direct updates the even pattern stops at $[0, 8]$ and the odd pattern cycles between $[0, K-1]$ and $[0, K]$. Since both patterns lie below F, their envelope equals F, with mean width 7.37 versus 15 for $\top$.

Model a's joined trajectory, 20 inputs in all, is

$$\bot \to [0,0] \to [0,2] \to [0,4] \to [0,5] \to [0,6] \to \cdots \to [0,20].$$

Model a has 6 omission inputs and 344 monotonicity violations, and model b has 4 and 291, respectively. All neural joined and direct runs terminate, and only

the planted odd pattern exhibits the stated direct cycle. Every trained model outputs the lower bound 0 on every nonempty input, as every training label does. A path consists of the inputs to which iteration applies a model, from $\perp$ to the final, stable input, each counted once. Paths have a median length of three inputs in both main-loop grids and four in the second loop, under 1% of the domain. Each unsound main-loop model stops at an input $[0, u]$ that it maps to itself, one short of F. This input is a training input for 21 of the 38 unsound models in the primary grid, 3 of the 18 in the $N = 50$ grid, and 7 of the 21 in the second loop.

Table 4 reports how much wider than F the envelopes are. For model a, the input $[0, 0]$ gives $F([0, 0]) = [0, 1]$ but $E([0, 0]) = T([0, 0]) = [0, 2]$. Extra width can also come from another input's prediction through the downward join.

Table 4. Envelope precision. Ranges of wider inputs are over models. For each model, $\overline{\Delta}$ averages $\text{width}(E(a)) - \text{width}(F(a))$ over the full domain.

N Cohort	Wider inputs	Median $\overline{\Delta}$	Range $\overline{\Delta}$
30 Trained	41–245	0.89	0.08–5.28
30 Retrained	7–245	0.29	0.01–5.28
50 Trained	73–667	2.51	0.06–4.25
50 Retrained	18–622	0.31	0.01–3.67

C Cost and Scale

The envelope looks up an offline table of all m oracle values, 497 and 1327 in the two grids. Retraining uses the same cached values for its offline checks, and the iteration of retrained models was replayed with F disabled. Runs used one thread for neural computation on an AMD Ryzen 7 7700 8-Core Processor, with 31 GiB visible memory. Software versions were Python 3.14.7 and PyTorch 2.14.1+cpu. Predictions are cached before pair checking. No reported training or retraining job failed or timed out. An audit reloaded the saved checkpoints of all reported models and retraining rounds and replayed neural iteration with oracle access disabled. No accepted result was unsound.

Across both grids, initial training takes 61.5 seconds and the complete retraining jobs take 39.7 seconds, including checks and controls but excluding startup and building the shared oracle cache.

The scale sweep in Table 5 uses $K = \lceil 0.6N \rceil$, including $K = 9$ at $N = 14$. At $N = 100$ and $K = 60$, F and the envelope prove 41/41 true upper-bound assertions, and $\top$ proves 1/41. Only sound results count as proofs.

Table 5. Complete scale sweep. Wall time includes the construction of the domain and its order, candidate and control checks (including repeated checks of the planted patterns), envelope construction, and assertion utility. It excludes startup and the independent audit. The width ratio is computed for the oracle F.

N	K	Inputs	Comparable pairs	Mean F width	$F/\top$	Wall (s)
14	9	121	3181	7.37	0.491	0.052
30	18	497	46873	13.72	0.443	0.714
50	30	1327	317578	22.37	0.439	5.06
100	60	5152	4603278	43.98	0.435	73.1
150	90	11477	22544603	65.58	0.434	360

D A Second Loop

The loop $x := 0$; **while** $x < K$ **do** $x := x + 2$ has the oracle $F(a) = [0, 0] \sqcup ((a \sqcap [0, K-1]) + 2)$. With $N = 30$ and $K = 18$, exact joined iteration reaches $[0, 19]$, an interval is sound if and only if it contains $[0, 18]$, and among sound intervals only $[0, 18]$ is rejected by the post-fixpoint check. We trained the 45 configurations of the primary grid with the same code, data partitions, and training procedure, after the main results and with the analysis plan fixed before training. Table 6 reports the outcomes. Unsound results occur between 58.1% and 79.7% held-out accuracy and sound ones between 43.2% and 91.9%, and 22 of the 24 sound results come from models that violate a global law. The three most accurate models (90.5–91.9%) violate both global laws yet reach the sound $[0, 19]$. The most accurate unsound model (79.7%) returns $[0, 11]$ on its training input $[0, 11]$, whose exact image is $[0, 13]$, and stops there. Of the 51 path omissions, the 30 whose upper bound is one short of F still exceed the current interval, so iteration continues, and the 21 that are two or three short stop it. Every model outputs $[0, 19]$ on $[0, 18]$, so no run stops at $[0, 18]$, and every envelope reaches $[0, 19]$. On the second loop, where a bound one short is harmless, the main loop's pattern reverses at widths 32 and 64: their 80%-coverage models give the most sound results, 4 and 5 of 5 (Table 6), whereas in the primary grid no model trained at 50% or 80% coverage is accepted.

Table 6. The second loop ($N = 30$, $K = 18$), by configuration. Accuracy is held-out exact match in percent. Every sound result is accepted by the post-fixpoint check. Omission share is the median percentage of inexact inputs that are omissions.

Width	Cov. (%)	Median	Range	Omission share (%)	Coverage pass	Global pass	Sound	Unsound
16	20	55.4	43.2–55.4	0	4/5	2/5	5	0
16	50	66.2	58.1–77.0	12	0/5	0/5	2	3
16	80	67.6	59.5–77.0	13	0/5	0/5	0	5
32	20	64.9	60.8–68.9	11	0/5	0/5	2	3
32	50	68.9	63.5–79.7	18	0/5	0/5	1	4
32	80	74.3	63.5–78.4	15	0/5	0/5	4	1
64	20	64.9	60.8–68.9	15	0/5	0/5	2	3
64	50	75.7	67.6–90.5	21	0/5	0/5	3	2
64	80	85.1	71.6–91.9	31	0/5	0/5	5	0
All		67.6	43.2–91.9	15	4/45	2/45	24	21